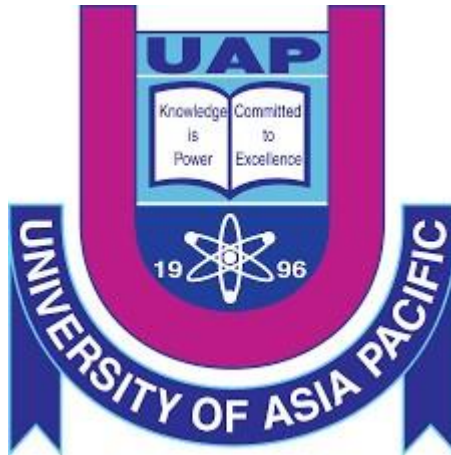




University of Asia Pacific

Department of Computer Science & Engineering
Database System Lab (CSE-212)

Inventory Management System for Emergency Supplies

Submitted By:

Team Name: *Inventory Innovators*

Team Information

Name	ID
Tanisha Taranoon Hridy	22201169
Hridita Lasker	22201184
Jannatul Fardaus Nirjum	22201193

Submitted To:

Alif Ruslan

Lecturer

Department of Computer Science & Engineering

University of Asia Pacific

Table of Contents

1. Topic -.....	3
2. Description of Topic-.....	3
3. Database Name-.....	3
4. Users-.....	3
5. Tables-.....	4
6. Attributes(Columns)-.....	5
7. Description of Each Table-.....	7
➤ Supplies-.....	7
➤ Stock-.....	8
➤ Locations-.....	8
➤ Suppliers-.....	8
➤ Supply_Orders-.....	9
➤ Emergency_Distributions-.....	9
➤ Emergency_events-.....	9
➤ Staff-.....	10
➤ Event_Supplies-.....	10
➤ Staff_event-.....	10
➤ Supplier_location-.....	10
8. Entity Relationship-.....	11
9.ER Diagram -.....	16
10. Schema Diagram-.....	17
11. Queries -.....	18
12. CEP mapping-.....	44
13. Reference	46

Project Name

Inventory Management System For Emergency Supplies

Description of Topic:-

An Inventory Management System for Emergency Supplies is specialized and focused on managing critical supplies needed in emergencies. We chose this topic intending to showcase a system that could significantly enhance crisis response efforts, potentially saving lives by improving the availability of essential supplies. This project will substantially impact real life by improving disaster response and emergency preparedness. It ensures the efficient management of critical supplies, reduces waste by tracking expiration dates, and allows for rapid deployment during crises. Maintaining accurate stock levels across multiple locations and providing real-time alerts for low supplies helps disaster response teams proactively.

DatabaseName:- Emergency_Supplies_Database

The database project intends to manage Critical supplies needed in emergencies, so we picked Emergency_Supplies_Database as our database name.

User:

The users of the “Inventory Management System for Emergency Supplies” would typically fall into several key categories, depending on the organization and its role in emergency response. The primary users of the Inventory Management System for Emergency Supplies include:

1. Emergency Response Coordinators

- Responsible for overseeing relief efforts and coordinating resource allocation.
- Use the system to identify supply availability and prioritize distribution to affected areas.

2. First Responders

- Firefighters, paramedics, police, and rescue teams operating on the ground.
- Use the system to quickly request or locate essential supplies, such as medical kits or rescue tools.

3. Medical Teams

- Doctors, nurses, and medical staff working in emergency shelters or mobile clinics.
- Use the system to track medical supplies like medications, first aid kits, and equipment.

4. Relief Workers

- Volunteers and field workers distributing food, water, and shelter materials to affected populations.
- Use the system to coordinate and report on distribution efforts.

5. Logistics Teams

- Personnel managing the transportation and delivery of supplies to emergency zones.
- Use the system to track inventory in transit and ensure timely delivery.

6. Government Emergency Planners

- Officials planning large-scale disaster relief operations.
- Use the system to monitor inventory levels, predict shortages, and plan for future emergencies.

These users rely on the system to improve the efficiency, speed, and effectiveness of emergency response operations, ultimately saving lives and reducing suffering during crises.

Table:

1. **Supplies**
2. **Locations**
3. **Suppliers**
4. **Stock**
5. **Supply_Orders**
6. **Emergency_Distributions**
7. **Emergency_events**
8. **Staff**
9. **Event_Supplies**
10. **Staff_Events**
11. **Supplier_Locations**

Attributes:

1. **Supplies**

- Supply_Id (**Primary Key**)
- Name
- Category
- Unit
- record_level

2. **Locations**

- Location_Id (**Primary key**)
- Name
- Address
- Region

3. **Suppliers**

- Suppliers_Id (**Primary key**)
- Name
- Contact_Info
- Lead_time

4. **Stock**

- Supply_Id (**Primary Key**)&(Foreign Key Referencing Supplies (Supply_Id))
- Location_Id(**Primary Key**)&(Foreign Key Referencing Locations (Location_Id))
- Quantity
- Expiration_date

5. **Supply_orders**

- Order_Id (**Primary Key**)
- Suppliers_Id (Foreign Key Referencing Suppliers (Suppliers_Id))
- Supply_Id (Foreign Key Referencing Supplies (Supply_Id))
- Order_date
- Quantity_ordered
- Status ('Pending','Received')

6. **Emergency_Distributions**

- Distribution_Id (**Primary Key**)
- Supply_Id (Foreign Key Referencing Supplies (Supply_Id))
- Location_Id(Foreign Key Referencing Locations (Location_Id))
- Quantity_dispatched
- Date_dispatched
- Emergency_type

7. **Staff**

- Staff_id (**Primary key**)
- Name
- Role
- Contact_info
- Assigned_location_id(**Foreign Key Referencing** Locations (Location_Id))

8. **Emergency_Events**

- event_id (**Primary key**)
- event_name
- start_date
- end_date
- event_type
- description

9. **Event_Supplies**

- event_id (**Primary Key**)&(**Foreign Key Referencing** Emergency_Events(event_id))
- supply_id (**Primary Key**)&(**Foreign Key Referencing** Supplies (Supply_Id))
- quantity_needed

10. **Staff_Events**

- staff_id (**Primary Key**)&(**Foreign Key Referencing** staff(staff_id))
- event_id (**Primary Key**)&(**Foreign Key Referencing** Emergency_Events(event_id))
- role

11. Supplier_Locations

- supplier_id(primary key)& (Foreign Key Referencing Suppliers (Suppliers_Id))
- location_id (Primary Key)&(Foreign Key Referencing Locations (Location_Id))
- delivery_frequency

Describe Of Each Table:

1. Supplies:

This table manages inventory details for different types of emergency supplies. *Supply_id* uniquely identify each supply item. And the other Attributes *Name,Category,Unit, & record_level* stores details about supplies,their categories,units of measurement and their thresholds for record.

Role: Tracks all items available for disaster response, ensuring proper categorization and inventory thresholds.

2. Locations:

This table stores details about warehouses ,depots or places where supplies are stored or dispatched.To uniquely identify each Location we use *location_Id* .And more location information is stored in *name,address & region attributes*.

Role: Tracks physical locations involved in storage, dispatch, and distribution of supplies

3. Suppliers

Manages Information about suppliers providing emergency supplies is the work of this table .Each supplier has a unique id as *Supplier_id* .*Name*, *contact_info* attributes are the information about the supplier, & *lead_time* attribute is the delivery lead times.

Role: Ensures reliable sourcing of supplies by maintaining supplier contact and delivery timelines.

4. Stock

This table maintains real-time inventory levels at different locations. Here, *supply_id* and *location_id* are composite keys linking supplies to a specific location. *Quantity* attributes indicate current stock levels, and *expiration_date* indicates the product's expiration date.

Role: Helps track where and how much of each supply is stored and identifies items approaching expiration.

5. Supply_Orders :

This table is for tracks orders placed with suppliers to replenish inventory . *Order_id* is used for the unique identifier for each order .*supply_id* & *supplier_id* links order to supply and supplies. *order_date*, *quantity_ordered*, & *status* tracks order timing, quantities, and fulfillment status.

Role: Manages procurement of supplies and monitors order progress.

6. Emergency_Events:

The purpose of this table is for catalogs details of disasters or emergencies requiring response. Every Event has a Unique identifier as *event_id*. *event_name, start_date, end_date, event_type, and description* each have information describing the nature and timeline of events.

Role: Provides context for resource allocation and response to specific emergencies.

7. Emergency_Distributions:

This table helps track the distribution of supplies to locations affected by emergencies. *distribution_id* is the unique identifier for each distribution record. *supply_id, location_id, and event_id* links supplies, locations. *emergencies.quantity_dispatched, date_dispatched, and emergency_type* are the details of dispatched supplies.

Role: Records and monitors supply dispatch during emergency responses.

8. Staff:

This table is for those who maintain details about personnel involved in managing supplies and operations. *staff_id* is a unique identifier for each staff member. Staff details and assigned working location are in the name, role, contact_info, and assigned_location_id attributes.

Role: Tracks human resources critical to inventory and emergency response operations.

9. Event_Supplies:

Helps in planning and resource allocation for emergency events. Acts as a blueprint for procurement or dispatching supplies to events.

Role: Tracks which supplies are required for each emergency event and the quantity needed

10. Staff_Events

Tracks personnel involvement in emergency responses. Assigns and organizes roles (e.g., leader, coordinator, logistician) for each event.

Role: Maps staff members to specific emergency events, along with their assigned roles.

11. Supplier_Locations

Ensures supplier assignments to specific warehouses or supply hubs. Maintains records of delivery schedules or frequencies. Optimizes logistics by linking suppliers to their designated delivery points.

Role: Tracks supplier relationships with specific locations, including delivery frequency or agreements.

Entity relationship:

1. Supplies & stock

Type of Relationship: **One-to-Many**

Details:

A single supply item (*supply_id*) can be stored in multiple locations, as tracked in Stock.

The *supply_id* in Stock is a *foreign key referencing Supplies(supply_id)*.

Purpose:

Tracks the inventory of each supply item across various locations.

2. Location & stock

Type of Relationship: One-to-Many

Details:

A single location (*location_id*) can store multiple supplies (*supply_id*), each tracked in the Stock table. The *location_id* in Stock is a *foreign key referencing Locations(location_id)*.

Purpose:

Links storage locations with their current inventory of supplies.

3. Supplies and Supply_Orders

Type of Relationship: One-to-Many

Details:

Each supply (*supply_id*) can be ordered multiple times from suppliers. The *supply_id* in Supply_Orders is a *foreign key referencing Supplies(supply_id)*.

Purpose:

Tracks specific supplies included in procurement orders.

4. Suppliers and Supply_Orders

Type of Relationship: **One-to-Many**

Details:

Each supplier (*supplier_id*) can receive multiple orders for supplies. The *supplier_id* in Supply_Orders is a *foreign key referencing Suppliers(supplier_id)*.

Purpose:

Tracks procurement of supplies from suppliers.

5. Supplies and Emergency_Distributions

Type of Relationship: **One-to-Many**

Details:

A single supply item (*supply_id*) can be distributed multiple times during various emergencies. The *supply_id* in Emergency_Distributions is a *foreign key referencing Supplies(supply_id)*.

Purpose:

Tracks the distribution of specific supplies for emergency responses.

6. Emergency_Events and Emergency_Distributions:

Type of Relationship: **One-to-Many**

Details:

A single emergency event (*event_id*) can involve multiple supply distributions. The *event_id* in Emergency_Distributions is a *foreign key referencing Emergency_Events(event_id)*.

Purpose:

Links specific emergency events to the supply distributions carried out to respond to them.

7. Staff and Locations

Type of Relationship: **One-to-Many**

Details:

A single location (*location_id*) can have multiple staff members assigned to it. The *assigned_location_id* in Staff is *a foreign key referencing Locations(location_id)*.

Purpose:

Associates staff members with their respective locations.

8. Locations and Emergency_Distributions

Type of Relationship: **One-to-Many**

Details:

A single location (*location_id*) can serve as the dispatch point for multiple distributions during emergencies. The *location_id* in Emergency_Distributions is *a foreign key referencing Locations(location_id)*.

Purpose:

Links distribution records to their originating locations.

9. Emergency_Events and Event_Supplies

Type of Relationship: **One-to-Many**

Details:

Each emergency event (*event_id*) can have a list of supplies needed.

The *event_id* in Event_Supplies is *a foreign key referencing Emergency_Events(event_id)*.

Purpose:

Tracks the specific supplies required for each emergency event.

10. Staff and Staff_Events

Type of Relationship: **One-to-Many**

Details:

Each staff member (*staff_id*) can participate in multiple emergency events. The *staff_id* in Staff_Events is a *foreign key referencing Staff(staff_id)*.

Purpose:

Assigns staff members to emergency events with specific roles.

11. Emergency_Events and Staff_Events

Type of Relationship: **One-to-Many**

Details:

Each emergency event (*event_id*) can involve multiple staff members. The *event_id* in Staff_Events is a *foreign key referencing Emergency_Events(event_id)*.

Purpose:

Tracks staff assignments for specific emergency events.

12. Suppliers and Supplier_Locations

Type of Relationship: **Many-to-Many**

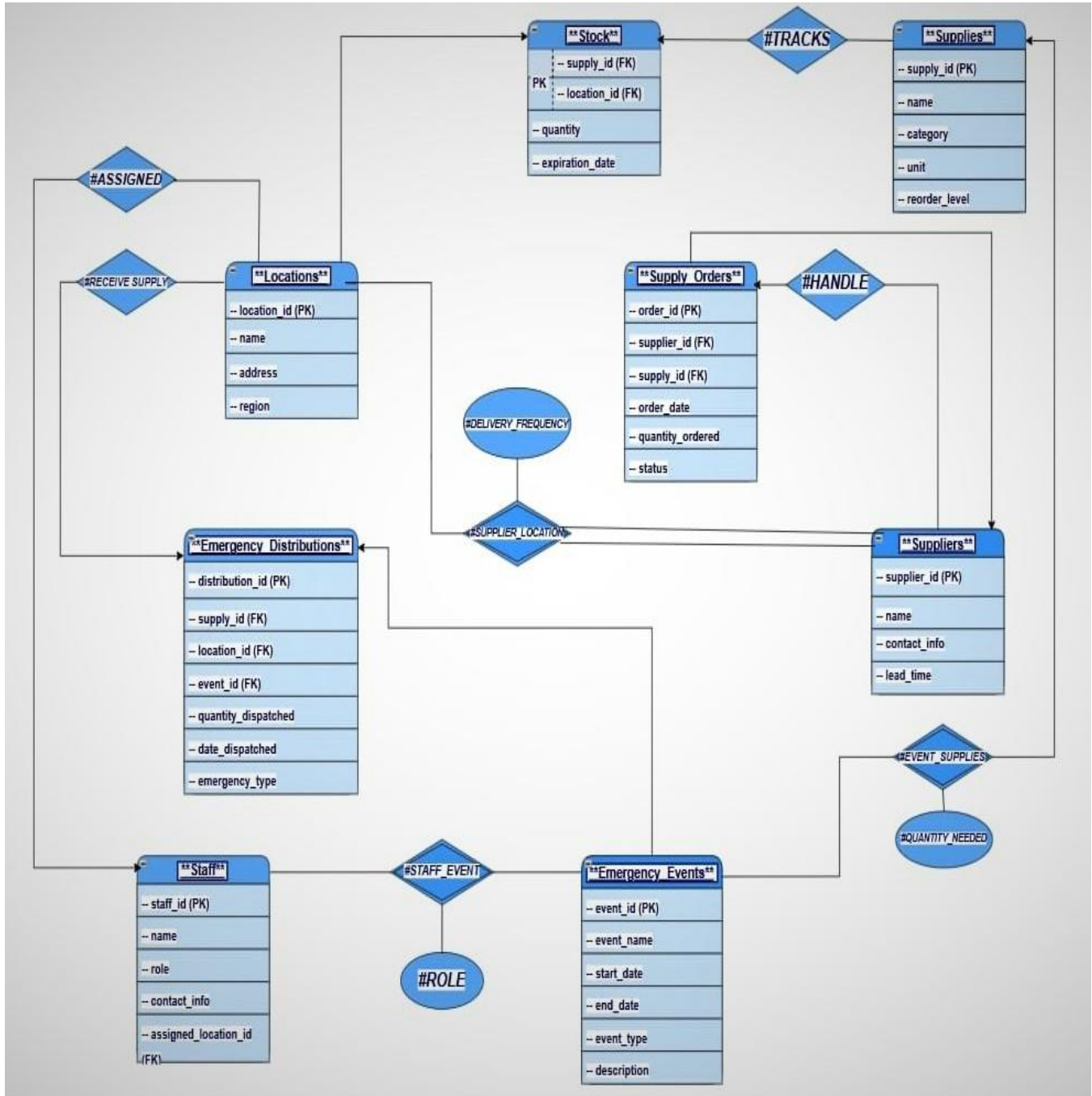
Details:

A supplier (*supplier_id*) can deliver to multiple locations, and a location (*location_id*) can receive deliveries from multiple suppliers. Implemented using the Supplier_Locations table.

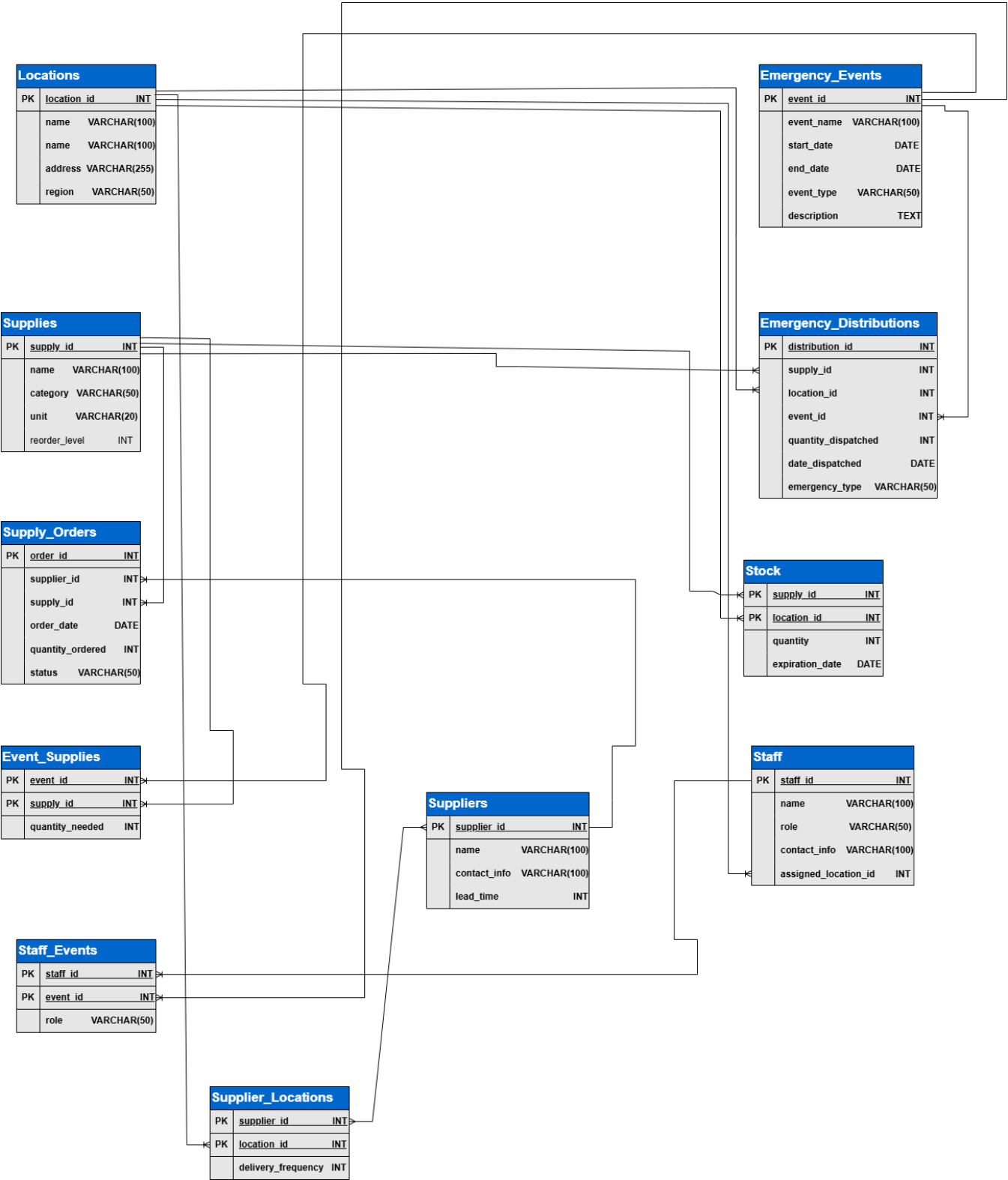
Purpose:

Tracks delivery capabilities between suppliers and locations.

ER Diagram:



Schema Diagram:



Implementing Queries:

Condition-based Queries (WHERE, BETWEEN, IN)

1. Query to list supplies with a reorder level less than 50:

Code:

```
SELECT name, reorder_level
FROM Supplies
WHERE reorder_level < 50;
```

Output:

name	reorder_level
Flashlights	30
Tents	10
Fire Extinguishers	20
Medicines	40
Emergency Radios	15

2. Retrieve supplies that start with the letter 'F':

Code:

```
SELECT name FROM Supplies WHERE name LIKE 'F%';
```

Output:

```

+-----+
| name          |
+-----+
| First Aid Kit  |
| Flashlights    |
| Fire Extinguishers |
+-----+

```

3. Query to find stock entries with quantities between 20 and 200:

Code:

```

SELECT supply_id, location_id, quantity
FROM Stock
WHERE quantity BETWEEN 20 AND 200;

```

Output:

```

+-----+-----+-----+
| supply_id | location_id | quantity |
+-----+-----+-----+
|          1 |          1 |       100 |
|          2 |          2 |       200 |
|          3 |          3 |        50 |
|          4 |          4 |       120 |
|          5 |          5 |        70 |
|          6 |          6 |        80 |
|          7 |          7 |        30 |
|          8 |          8 |        40 |
|          9 |          9 |        60 |
|         10 |         10 |        20 |
+-----+-----+-----+

```

4. Find emergency events that happened after 2020:

Code:

```

SELECT event_name, start_date
FROM Emergency_Events
WHERE start_date > '2020-01-01';

```

Output:

event_name	end_date
Hurricane Relief	2024-11-10
Earthquake Assistance	2024-10-30
Flood Recovery	2024-09-25
Wildfire Response	2024-08-15
Pandemic Support	2024-07-31
Drought Assistance	2024-06-20
Refugee Support	2024-05-15
Storm Relief	2024-04-10
Tsunami Relief	2024-03-10
Fire Evacuation Support	2024-02-15

5. Query to fetch emergency events of specific types:

Code:

```

SELECT event_id, event_name, event_type
FROM Emergency_Events
WHERE event_type IN ('Natural Disaster', 'Health Emergency');

```

Output :

event_id	event_name	event_type
1	Hurricane Relief	Natural Disaster
2	Earthquake Assistance	Natural Disaster
3	Flood Recovery	Natural Disaster
4	Wildfire Response	Natural Disaster
6	Drought Assistance	Natural Disaster
8	Storm Relief	Natural Disaster
9	Tsunami Relief	Natural Disaster

6. List categories with an average reorder level greater than 50:

Code:

```
SELECT category, AVG(reorder_level) AS "Average Reorder Level"
FROM Supplies
GROUP BY category
HAVING AVG(reorder_level) > 50;
```

Output:

category	Average Reorder Level
Beverage	200.0000
Shelter	55.0000
Food	150.0000

Aggregated Queries

7. Count the total number of supplies:

Code:

```
SELECT COUNT(*) AS "Total Supplies" FROM Supplies;
```

Output:

```
+-----+
| Total Supplies |
+-----+
|           10 |
+-----+
```

8. Calculate the average reorder level by category:

Code:

```
SELECT category, AVG(reorder_level) AS "Average Reorder Level"  
FROM Supplies GROUP BY category;
```

Output :

category	Average Reorder Level
Medical	45.0000
Beverage	200.0000
Shelter	55.0000
Food	150.0000
Lighting	30.0000
Power	50.0000
Safety	20.0000
Communication	15.0000

9. Find Supplies That Are Below the Average Reorder Level

Code:

```
SELECT name, reorder_level FROM Supplies
WHERE
    reorder_level < (SELECT AVG(reorder_level) FROM Supplies);
```

output:

name	reorder_level
First Aid Kit	50
Flashlights	30
Batteries	50
Tents	10
Fire Extinguishers	20
Medicines	40
Emergency Radios	15

10. Total dispatched supplies for an event:

Code:

```
SELECT event_id, SUM(quantity_dispatched) AS "Total Dispatched"
FROM Emergency_Distributions
GROUP BY event_id;
```

Output :

event_id	Total Dispatched
1	20
2	100
3	50
4	80
5	30
6	40
7	10
8	25
9	60
10	15

11. Query to count the number of supplies in each category:

Code:

```
SELECT category, COUNT(*) as supply_count
FROM Supplies GROUP BY category
ORDER BY supply_count DESC;
```

Output :

category	supply_count
Medical	2
Shelter	2
Beverage	1
Food	1
Lighting	1
Power	1
Safety	1
Communication	1

12. Sort supplies by name in ascending order:

Code:

SELECT * FROM Supplies ORDER BY name ASC;

Output:

supply_id	name	category	unit	reorder_level
6	Batteries	Power	Pack	50
3	Blankets	Shelter	Piece	100
4	Canned Food	Food	Cans	150
10	Emergency Radios	Communication	Piece	15
8	Fire Extinguishers	Safety	Piece	20
1	First Aid Kit	Medical	Box	50
5	Flashlights	Lighting	Piece	30
9	Medicines	Medical	Box	40
7	Tents	Shelter	Piece	10
2	Water Bottles	Beverage	Liters	200

13. Retrieve the name of the supply with the highest price

Code:

```
SELECT name, unit AS "Price"
FROM Supplies
WHERE unit = ( SELECT MAX(unit)
FROM Supplies);
```

Output:

name	Price
Blankets	Piece
Flashlights	Piece
Tents	Piece
Fire Extinguishers	Piece
Emergency Radios	Piece

14. Find the most recently added supply

Code:

```
SELECT supply_id, name
FROM Supplies
WHERE supply_id = ( SELECT MAX(supply_id)
FROM Supplies);
```

Output:

supply_id	name
10	Emergency Radios

15. To Get the earliest and latest stock expired dates:

Code:

```
SELECT MIN(expiry_date) AS earliest_expiry_date,  
       MAX(expiry_date) AS latest_expiry_date  
FROM Stock;
```

Output:

supply_id	name
10	Emergency Radios

Join Queries

16. List all supplies distributed during emergency events:

Code:

```
SELECT ee.event_name,s.name as "Supply Name"  
      ,ed.quantity_dispatched FROM Emergency_Distributions ed  
      JOIN Emergency_Events ee ON ed.event_id = ee.event_id  
      JOIN Supplies s ON ed.supply_id = s.supply_id;
```

Output:

event_name	Supply Name	quantity_dispatched
Hurricane Relief	First Aid Kit	20
Earthquake Assistance	Water Bottles	100
Flood Recovery	Blankets	50
Wildfire Response	Canned Food	80
Pandemic Support	Flashlights	30
Drought Assistance	Batteries	40
Refugee Support	Tents	10
Storm Relief	Fire Extinguishers	25
Tsunami Relief	Medicines	60
Fire Evacuation Support	Emergency Radios	15

17. Retrieve supplies and their current stock level for each location:

Code:

```
SELECT l.name AS "Location", s.name AS "Supply", st.quantity
FROM Stock st
      JOIN Supplies s ON st.supply_id = s.supply_id
      JOIN Locations l ON st.location_id = l.location_id;
```

Output:

Location	Supply	quantity
Central Warehouse	First Aid Kit	100
East Storage	Water Bottles	200
West Depot	Blankets	50
South Hub	Canned Food	120
North Facility	Flashlights	70
Midtown Storage	Batteries	80
City Center Warehouse	Tents	30
Emergency Depot	Fire Extinguishers	40
Regional HQ	Medicines	60
Coastal Facility	Emergency Radios	20

18. Retrieve staff assignments for specific emergency events:

Code:

```
SELECT ee.event_name, st.name AS "Staff Name", sa.role
FROM Staff_Events sa
      JOIN Staff st ON sa.staff_id = st.staff_id
      JOIN Emergency_Event ee ON sa.event_id = ee.event_id;
```

Output:

event_name	Staff Name	role
Hurricane Relief	John Smith	Coordinator
Earthquake Assistance	Alice Brown	Supervisor
Flood Recovery	Bob Johnson	Technician
Wildfire Response	Emily Davis	Inventory Manager
Pandemic Support	Michael Wilson	Team Lead
Drought Assistance	Jessica Taylor	Driver
Refugee Support	Chris White	Warehouse Worker
Storm Relief	Sarah Hall	Emergency Responder
Tsunami Relief	David Clark	Procurement Specialist
Fire Evacuation Support	Anna Moore	Administrator

19. Emergency Events Needing the Most Staff:

Code:

```
SELECT ee.event_name, COUNT(se.staff_id) AS "Total Staff Assigned"
FROM Staff_Events se
JOIN Emergency_Events ee ON se.event_id = ee.event_id
GROUP BY ee.event_name
ORDER BY COUNT(se.staff_id) DESC;
```

Output:

event_name	Total Staff Assigned
Hurricane Relief	1
Earthquake Assistance	1
Flood Recovery	1
Wildfire Response	1
Pandemic Support	1
Drought Assistance	1
Refugee Support	1
Storm Relief	1
Tsunami Relief	1
Fire Evacuation Support	1

20. Retrieve Emergency Events with Dispatched Supplies(Inner Join):

Code:

```
SELECT ee.event_name, s.name AS Supply_Name,  
ed.quantity_dispatched FROM Emergency_Events ee  
INNER JOIN  
Emergency_Distributions ed ON ee.event_id = ed.event_id  
INNER JOIN  
Supplies s ON ed.supply_id = s.supply_id;
```

Output:

event_name	Supply_Name	quantity_dispatched
Hurricane Relief	First Aid Kit	20
Earthquake Assistance	Water Bottles	100
Flood Recovery	Blankets	50
Wildfire Response	Canned Food	80
Pandemic Support	Flashlights	30
Drought Assistance	Batteries	40
Refugee Support	Tents	10
Storm Relief	Fire Extinguishers	25
Tsunami Relief	Medicines	60
Fire Evacuation Support	Emergency Radios	15

21. List All Suppliers and Their Supply Orders

Code:

```
SELECT sp.name AS Supplier_Name, so.order_id, so.order_date,  
so.quantity_ordered  
FROM Suppliers sp  
LEFT JOIN Supply_Orders so ON sp.supplier_id = so.supplier_id  
ORDER BY sp.name, so.order_date;
```

Output:

Supplier_Name	order_id	order_date	quantity_ordered
Food Essentials	3	2024-09-20	150
Global Distributors	9	2024-10-15	70
Global Health Supplies	1	2024-11-01	50
Light and Power	4	2024-11-05	100
MediSupply	7	2024-10-30	30
North Supply Chain	10	2024-11-20	80
RadioComm Ltd.	8	2024-11-10	5
Safety First Inc.	6	2024-09-25	20
TentPro	5	2024-10-10	10
WaterWorks Co.	2	2024-10-15	300

Advanced Queries:

22. Update salaries of staff based on their role:

Code:

Alter **table** Staff

add column salary decimal(10,3);

UPDATE Staff

SET salary = **CASE**

WHEN role = 'Manager' **THEN** 10000.50

WHEN role = 'Field Coordinator' **THEN** 2000.00

WHEN role = 'Medical Specialist' **THEN** 3000.50

WHEN role = 'Logistics Officer' **THEN** 2500

ELSE 1000

END;

```

UPDATE Staff
SET salary = CASE
    WHEN role = 'Manager' THEN salary * 1.10
    ELSE salary * 1.05
END;
SELECT * from Staff;

```

Output:

staff_id	name	role	contact_info	assigned_location_id	salary
1	John Smith	Manager	john.smith@example.com	1	NULL
2	Alice Brown	Logistics Coordinator	alice.brown@example.com	2	NULL
3	Bob Johnson	Technician	bob.johnson@example.com	3	NULL
4	Emily Davis	Inventory Specialist	emily.davis@example.com	4	NULL
5	Michael Wilson	Supervisor	michael.wilson@example.com	5	NULL
6	Jessica Taylor	Driver	jessica.taylor@example.com	6	NULL
7	Chris White	Warehouse Worker	chris.white@example.com	7	NULL
8	Sarah Hall	Emergency Response	sarah.hall@example.com	8	NULL
9	David Clark	Procurement Officer	david.clark@example.com	9	NULL
10	Anna Moore	Administrator	anna.moore@example.com	10	NULL
staff_id	name	role	contact_info	assigned_location_id	salary
1	John Smith	Manager	john.smith@example.com	1	3000.500
2	Alice Brown	Logistics Coordinator	alice.brown@example.com	2	2500.000
3	Bob Johnson	Technician	bob.johnson@example.com	3	2500.000
4	Emily Davis	Inventory Specialist	emily.davis@example.com	4	7000.000
5	Michael Wilson	Supervisor	michael.wilson@example.com	5	5000.000
6	Jessica Taylor	Driver	jessica.taylor@example.com	6	2500.000
7	Chris White	Warehouse Worker	chris.white@example.com	7	10000.500
8	Sarah Hall	Emergency Response	sarah.hall@example.com	8	2500.000
9	David Clark	Procurement Officer	david.clark@example.com	9	2500.000
10	Anna Moore	Administrator	anna.moore@example.com	10	2500.000
staff_id	name	role	contact_info	assigned_location_id	salary
1	John Smith	Manager	john.smith@example.com	1	3300.550
2	Alice Brown	Logistics Coordinator	alice.brown@example.com	2	2625.000
3	Bob Johnson	Technician	bob.johnson@example.com	3	2625.000
4	Emily Davis	Inventory Specialist	emily.davis@example.com	4	7350.000
5	Michael Wilson	Supervisor	michael.wilson@example.com	5	5250.000
6	Jessica Taylor	Driver	jessica.taylor@example.com	6	2625.000
7	Chris White	Warehouse Worker	chris.white@example.com	7	10500.525
8	Sarah Hall	Emergency Response	sarah.hall@example.com	8	2625.000
9	David Clark	Procurement Officer	david.clark@example.com	9	2625.000
10	Anna Moore	Administrator	anna.moore@example.com	10	2625.000

23. Find events and their total dispatched supply cost:

Code:

```
SELECT ee.event_name, SUM(ed.quantity_dispatched * s.unit) AS "Total  
Cost"  
FROM Emergency_Distributions ed  
JOIN Emergency_Events ee ON ed.event_id = ee.event_id  
JOIN Supplies s ON ed.supply_id = s.supply_id  
GROUP BY ee.event_name;
```

Output:

event_name	Total Cost
Hurricane Relief	0
Earthquake Assistance	0
Flood Recovery	0
Wildfire Response	0
Pandemic Support	0
Drought Assistance	0
Refugee Support	0
Storm Relief	0
Tsunami Relief	0
Fire Evacuation Support	0

24. Check if a supply has been dispatched during any emergency event:

Code:

```
SELECT name,CASE  
WHEN EXISTS (SELECT 1 FROM Emergency_Distributions ed  
WHERE ed.supply_id = s.supply_id)THEN 'Dispatched'  
ELSE 'Not Dispatched'  
END AS status FROM Supplies s;
```

Output:

name	status
First Aid Kit	Dispatched
Water Bottles	Dispatched
Blankets	Dispatched
Canned Food	Dispatched
Flashlights	Dispatched
Batteries	Dispatched
Tents	Dispatched
Fire Extinguishers	Dispatched
Medicines	Dispatched
Emergency Radios	Dispatched

25. Retrieve Supplies with the Highest Quantity in Each Category

Code:

```
SELECT s1.name AS Supply_Name, s1.category, s1.reorder_level
FROM Supplies s1 WHERE
s1.reorder_level = (SELECT MAX(s2.reorder_level)
FROM Supplies s2 WHERE s2.category = s1.category)
ORDER BY s1.category;
```

Output:

Supply_Name	category	reorder_level
Water Bottles	Beverage	200
Emergency Radios	Communication	15
Canned Food	Food	150
Flashlights	Lighting	30
First Aid Kit	Medical	50
Batteries	Power	50
Fire Extinguishers	Safety	20
Blankets	Shelter	100

26. Calculate Total Supplies Dispatched per Event

Code:

```
SELECT ee.event_name,  
       SUM(ed.quantity_dispatched) AS Total_Dispatched  
FROM   Emergency_Events ee  
INNER JOIN  
       Emergency_Distributions ed ON ee.event_id = ed.event_id  
GROUP BY ee.event_name  
ORDER BY Total_Dispatched DESC;
```

Output:

event_name	Total_Dispatched
Earthquake Assistance	100
Wildfire Response	80
Tsunami Relief	60
Flood Recovery	50
Drought Assistance	40
Pandemic Support	30
Storm Relief	25
Hurricane Relief	20
Fire Evacuation Support	15
Refugee Support	10

27. List Suppliers and the Number of Supplies They Provide

Code:

```
SELECT sp.name AS Supplier_Name, l.name AS Location_Name,  
       COUNT(sl.location_id) AS Locations_Served  
FROM Suppliers sp  
INNER JOIN  
    Supplier_Locations sl ON sp.supplier_id = sl.supplier_id  
INNER JOIN  
    Locations l ON sl.location_id = l.location_id  
GROUP BY sp.name, l.name  
ORDER BY Locations_Served DESC;
```

Output:

Supplier_Name	Location_Name	Locations_Served
Global Health Supplies	Central Warehouse	1
WaterWorks Co.	East Distribution Center	1
Food Essentials	West Distribution Center	1
Light and Power	South Regional Hub	1
TentPro	North Regional Hub	1
Safety First Inc.	Overseas Depot	1
MediSupply	Southwest Supply Point	1
RadioComm Ltd.	Northeast Fulfillment	1
Global Distributors	Mountain Warehouse	1
North Supply Chain	West Coast Distribution	1

28. Find the Total Quantity of Each Supply Dispatched Across

All Events

Code :

```
SELECT s.name AS Supply_Name,  
       SUM(ed.quantity_dispatched) AS Total_Dispatched  
FROM Supplies s Join  
     Emergency_Distributions ed ON s.supply_id = ed.supply_id  
GROUP BY s.name ORDER BY Total_Dispatched ;
```

Output:

Supply_Name	Total_Dispatched
Tents	10
Emergency Radios	15
First Aid Kit	20
Fire Extinguishers	25
Flashlights	30
Batteries	40
Blankets	50
Medicines	60
Canned Food	80
Water Bottles	100

29. List Suppliers and Their Average Lead Time for Supplies They Provide

Code:

```
SELECT sp.name AS Supplier_Name,  
       AVG(sp.lead_time) AS Average_Lead_Time  
FROM Suppliers sp  
JOIN Supply_Orders so ON sp.supplier_id = so.supplier_id  
GROUP BY sp.name  
ORDER BY Average_Lead_Time ASC;
```

Output:

Supplier_Name	Average_Lead_Time
North Supply Chain	4.0000
Food Essentials	5.0000
Safety First Inc.	6.0000
Global Health Supplies	7.0000
MediSupply	7.0000
Light and Power	8.0000
Global Distributors	9.0000
WaterWorks Co.	10.0000
TentPro	12.0000
RadioComm Ltd.	15.0000

30. Regions With Highest Emergency Distributions

Code:

```
SELECT l.region,  
       SUM(ed.quantity_dispatched) AS Total_Dispatched  
FROM Emergency_Distributions ed  
     JOIN Locations l ON ed.location_id = l.location_id  
     GROUP BY l.region ORDER BY Total_Dispatched DESC;
```

Output:

region	Total_Dispatched
Northeast	125
Southeast	80
West	65
Mountain	60
Midwest	50
Pacific	40
Southwest	10

31. List Locations with Stock Levels Below Critical Thresholds

Code:

```
SELECT s.location_id,s.supply_id,s.quantity,sp.reorder_level  
FROM Stock s JOIN Supplies sp ON s.supply_id = sp.supply_id  
WHERE s.quantity < sp.reorder_level;
```

Output:

location_id	supply_id	quantity	reorder_level
3	3	50	100
4	4	120	150

32. Predict Future Supply Needs Based on Past Usage Data

Output:

```
SELECT supply_id,  
       AVG(quantity_dispatched) AS avg_quantity_dispatched  
FROM Emergency_Distributions GROUP BY supply_id;
```

Output:

supply_id	avg_quantity_dispatched
1	20.0000
2	100.0000
3	50.0000
4	80.0000
5	30.0000
6	40.0000
7	10.0000
8	25.0000
9	60.0000
10	15.0000

Simple Queries

33. Natural join

Code:

```
select * from supplies Natural join Stock;
```

output:

supply_id	name	category	unit	reorder_level	location_id	quantity	expiration_date
1	First Aid Kit	Medical	Box	50	1	100	2025-01-01
2	Water Bottles	Beverage	Liters	200	2	200	2024-12-01
3	Blankets	Shelter	Piece	100	3	50	NULL
4	Canned Food	Food	Cans	150	4	120	2025-03-01
5	Flashlights	Lighting	Piece	30	5	70	2024-11-01
6	Batteries	Power	Pack	50	6	80	2025-06-01
7	Tents	Shelter	Piece	10	7	30	NULL
8	Fire Extinguishers	Safety	Piece	20	8	40	2026-01-01
9	Medicines	Medical	Box	40	9	60	2024-10-01
10	Emergency Radios	Communication	Piece	15	10	20	2025-08-01

34. Add a Column

Code:

```
ALTER TABLE Staff
```

```
ADD COLUMN hire_date DATE;
```

```
Select * from staff;
```

Output:

staff_id	name	role	contact_info	assigned_location_id	salary	hire_date
1	John Smith	Manager	john.smith@example.com	1	3300.550	NULL
2	Alice Brown	Logistics Coordinator	alice.brown@example.com	2	2625.000	NULL
3	Bob Johnson	Technician	bob.johnson@example.com	3	2625.000	NULL
4	Emily Davis	Inventory Specialist	emily.davis@example.com	4	7350.000	NULL
5	Michael Wilson	Supervisor	michael.wilson@example.com	5	5250.000	NULL
6	Jessica Taylor	Driver	jessica.taylor@example.com	6	2625.000	NULL
7	Chris White	Warehouse Worker	chris.white@example.com	7	10500.525	NULL
8	Sarah Hall	Emergency Response	sarah.hall@example.com	8	2625.000	NULL
9	David Clark	Procurement Officer	david.clark@example.com	9	2625.000	NULL
10	Anna Moore	Administrator	anna.moore@example.com	10	2625.000	NULL

35. Delete a Column

```
ALTER TABLE Suppliers
```

```
DROP COLUMN contact_info;
```

Output:

supplier_id	name	lead_time
1	Global Health Supplies	7
2	WaterWorks Co.	10
3	Food Essentials	5
4	Light and Power	8
5	TentPro	12
6	Safety First Inc.	6
7	MediSupply	7
8	RadioComm Ltd.	15
9	Global Distributors	9
10	North Supply Chain	4

36. Update an Existing Record

Code:

UPDATE Supplies

SET reorder_level = 25

WHERE name = 'First Aid Kits';

Output:

supply_id	name	category	unit	reorder_level
1	First Aid Kit	Medical	Box	50
2	Water Bottles	Beverage	Liters	200
3	Blankets	Shelter	Piece	100
4	Canned Food	Food	Cans	150
5	Flashlights	Lighting	Piece	30
6	Batteries	Power	Pack	50
7	Tents	Shelter	Piece	10
8	Fire Extinguishers	Safety	Piece	20
9	Medicines	Medical	Box	40
10	Emergency Radios	Communication	Piece	15

CEP Mapping:

How Knowledge Profiles (K's) are addressed through the project and mapping among K's, COs, and POs

K's	Attributes	How K's are addressed through our project	CO	PO
K3	Engineering Fundamentals	Our project uses engineering fundamentals such as programming, database design, and SQL to implement the inventory system effectively.	CO1, CO2	PO1, PO2
K4	Specialist Knowledge	Since our project focuses on emergency supplies, it required specialized knowledge of supply chain logistics and disaster management	CO3, CO6	PO4, PO6
K5	Engineering Design	Entity-Relationship Diagrams and Schema Diagrams were created for a systematic database design process.	CO5, CO4	PO3, PO2
K6	Engineering Practice	The database was implemented in MySQL Online using DDL and DML commands. Queries for stock monitoring, replenishment, and expiration tracking were developed.	CO5, CO7	PO1, PO6

K7	Comprehension of Engineering in Society	The project ensures societal benefit by improving disaster response readiness through effective inventory management and allocation.	CO6,CO8	PO6, PO8
----	---	--	---------	----------

How Complex Engineering Problem Solving (P's) are addressed through the project and mapping among P's

P'S	Attributes	How P's are addressed through our project	CO	PO
P1	Depth of knowledge required	The project required in-depth understanding of SQL queries, and emergency supply chain challenges.	CO1, CO2	PO1, PO2
P2	Range of conflicting requirements	Conflicting demands like minimizing stock wastage while ensuring sufficient supply were managed using systematic inventory checks	CO3, CO6	PO3, PO5
P4	Familiarity of issues	The system addresses rarely encountered issues such as inventory shortages during unpredictable emergencies	CO2, CO4	PO1, PO4
P7	Interdependence	The system integrates stock tracking, expiration monitoring, and replenishment planning, which are interdependent modules	CO2, CO3	PO5, PO7

How Complex Engineering Activities (A's) are addressed through the project and mapping among A's, COs, and Pos:

A's	Attributes	How A's are addressed through our project	CO	PO
A1	Range of resources	Developers utilized SQL tools, database platforms, and inventory management principles for efficient emergency supply tracking.	CO1	PO1,PO3
A4	Consequences for society and the environment	The system ensures efficient disaster response, minimizes waste, and improves resource allocation during emergencies.	CO6,CO7	PO6,PO8
A5	Familiarity	Familiarity with emergency logistics, disaster response requirements, and inventory systems was key to addressing challenges	CO8,CO9	P10,P11

References:

- Database System Concepts- by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan (7th edition)
- Database System Concepts- by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan(6th edition)

- Draw.io
- <https://youtu.be/obb7SIUmKQE?si=WvMG6Y-bvgHtULSr>
- Database Management Systems (DBMS) Playlist By Farhan Hossian
- Grammarly
- Gemini
- Previous semester's Report (Senior Sarif & Senior Priom)
- Stored in Drive
<https://drive.google.com/drive/folders/1Xr05BX9C-ftQOQeRY6y-yX8YiwRHqwNo>